

TD 4 : Thread, Lock

Objectifs pédagogiques :

- thread
- atomic, mutex
- section critique

Introduction

Dans ce TD, nous allons aborder l'utilisation des thread en C++ pour faire de la programmation concurrente. L'exercice permet de voir plusieurs mécanismes que l'on peut employer pour protéger les données critiques des accès concurrent.

Pour être sûr de ne pas entrer en conflit avec d'autres applications, nous utiliserons le namespace `pr` pour notre implémentation.

1.1 Création de Thread

On considère une fonction `void work(int id)` qui affiche son identifiant `id`, puis dort pendant une durée aléatoire comprise entre 0 et 1000 ms, puis affiche un deuxième message.

```
                                work
1  void work (int index) {
2      std::cout << "started " << index << endl;
3      auto r = ::rand() % 1000 ; // 0 to 1 sec
4      std::this_thread::sleep_for (std::chrono::milliseconds(r));
5      std::cout << "finished " << index << endl;
6  }
```

Question 1. Ecrire une fonction `void createAndWait(int N)` qui crée N threads exécutant `work` avec pour identifiant leur ordre de création compris entre 0 et $N - 1$, puis attend qu'ils soient tous terminés. On souhaite maximiser la concurrence.

On doit pouvoir l'utiliser avec le main suivant :

```
                                work
1  int main (int argc, const char ** argv) {
2      int N = 3;
3      if (argc > 1) {
4          N=atoi(argv[1]);
5      }
6      // pour éviter des executions trop reproductibles, pose le seed.
7      ::srand(::time(nullptr));
8      return pr::createAndWait(N);
9  }
```

On note :

Attention à `rand` pseudo aléatoire, utiliser `srand(time)` permet d'assurer que les exécutions varient un peu.

La partie "maximiser la concurrence" ça veut dire créer tout le monde avant le premier `join`. On peut discuter la boucle qui ferait : `for (int i<N) { thread t (work,i); t.join() }`

Elle crée bien N thread mais sans concurrence possible.

J'ai fait `emplace_back` ici mais `push_back` irait bien aussi (on donne les deux syntaxes).

```

                                createjoin.cpp
1  int createAndWait (int N) {
2      vector<thread> threads;
3      threads.reserve(N);
4      for (int i=0; i < N ; i++) {
5          // push_back, plus explicite sur l'invocation du ctor de thread
6          // threads.push_back(thread(work,i));
7          // NB : emplace_back forward les arguments au constructeur de thread
8          threads.emplace_back(work,i);
9          std::cout << "created "<< i << endl;
10     }
11     for (int i=0; i < N ; i++) {
12         threads[i].join();
13         std::cout << "joined "<< i << endl;
14     }
15     return 0;
16 }

```

Question 2. Si l'on exécute le programme, quels sont les entrelacements possibles ?

Donc ce qu'on sait :

- chaque thread exécute ses instructions dans l'ordre (sauf si le matériel/compilo s'en mêle avec out of order execution. Mais n'en parlons pas à ce stade.)
- les créations par le père précèdent le début des exécutions des fils
- les terminaisons par les fils précèdent le join du père
- les autres instructions ne sont pas comparables/ordonnées

Le dessin suivant exhibe ça :

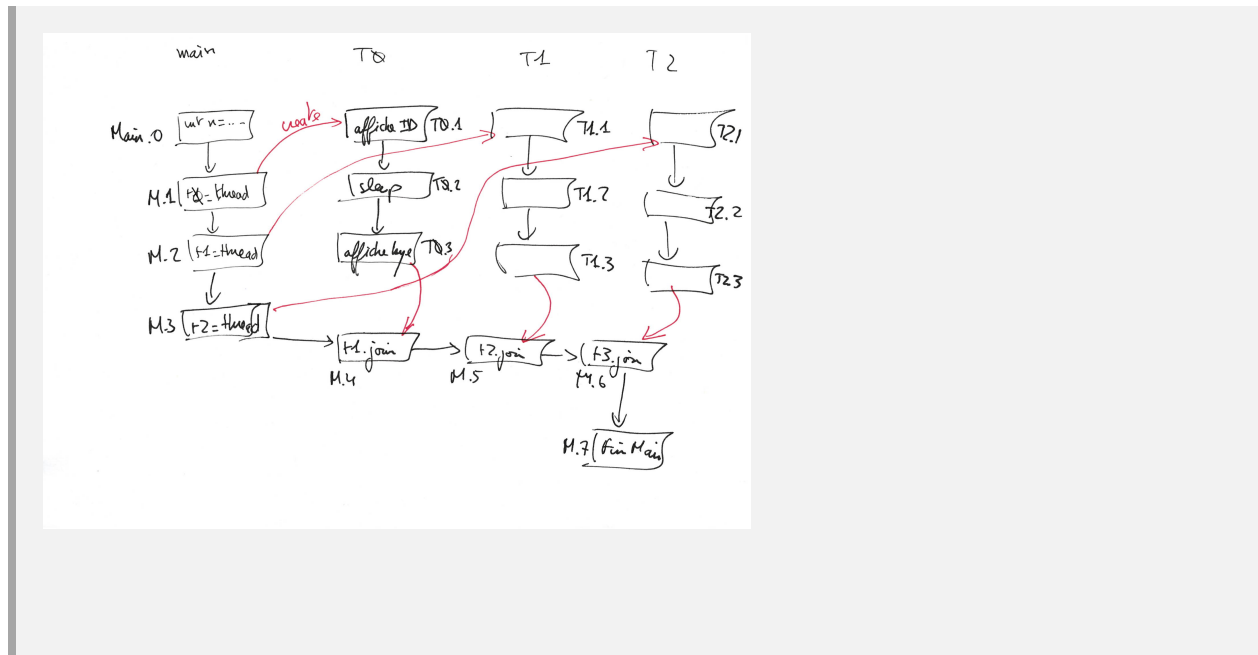
- en noir les actions liées au sein du code d'un thread
- en rouge les synchronisations inter-thread

On dessine ici pour trois threads créés, chacun ayant trois instructions : "mon id est ...", "sleep(rand)", "id XXX se finit"

Le main crée puis join les threads.

On remarque un *ordre partiel* : il est impossible de déterminer si $T0.1$ précède $T1.1$ par exemple. Les deux actions seront respectivement après M1 et après M2 (deux actions qui sont ordonnées au sein du main) mais leur ordre n'est pas fixé par le programme en l'état.

Ce dessin permet de capturer tous les entrelacements possibles si on le lit correctement.



Question 3. Si on ajoute un affichage “created id” juste après l’invocation du constructeur de thread, et un affichage “joined id” juste après l’appel à `join`. Que peut on dire, pour un identifiant donné, de l’ordre entre les messages : created/started/finished/join ?

Réponse :

- started précède finished (ordonné dans fils).
- created précède joined (ordonné dans pere).
- finished précède joined (sémantique join = synchro)

MAIS created est incomparable à started et finished, on peut observer c,s,f ou s,c,f ou s,f,c.

En particulier on peut voir passer finished avant created !

1.2 Variables partagées.

On considère une classe `Compte` munie d’un attribut entier représentant le solde, d’un constructeur positionnant le solde initial, d’une opération membre `void crediter (int val)` qui ajoute de l’argent au solde, et d’une opération membre `int getSolde() const`.

On donne aussi une fonction `void jackpot (Compte & c)` qui crédite 10000 pièces d’or sur le compte fourni, mais une par une (ding, ding, ding...).

jackpot.cpp

```

1 #include <vector>
2 #include <thread>
3 using namespace std;
4
5 class Compte {
6     int solde;
7 public :
8     Compte(int solde=0):solde(solde) {}
9     void crediter (size_t val) { solde+=val ;}
10    int getSolde() const {return solde;}
11 };
12 const int JP = 10000;
  
```

```

13 void jackpot(Compte & c) {
14     for (int i=0; i < JP; i++)
15         c.crediter(1);
16 }
17
18 const int NB_THREAD = 10;
19 int main () {
20     vector<thread> threads;
21     // TODO : creer des threads qui font jackpot sur un compte
22     for (auto & t : threads) {
23         t.join();
24     }
25     // TODO : tester solde = NB_THREAD * JP
26     return 0;
27 }

```

Question 4. Complétez le main, de façon à instancier un compte, et créer N threads qui exécutent le code de la fonction Jackpot.

On instancie la variable partagée AVANT de créer les thread pour permettre de leur passer.

On note le `std::ref` pour garantir quand on passe la réf du compte (à la création des thread) qu'elle persistera pendant la vie du thread. C'est le programmeur qui fait cette garantie (attention à ne pas mentir !).

Si par exemple, la boucle de join était faite après un bloc qui déclare l'instance de Compte, le comportement serait incorrect.

```

1 vector<thread> threads;
2 { // bloc
3     Compte c;
4     for (int i...) threads.emplace_back(work, std::ref(c));
5 } // fin bloc (BUG)
6 for (auto & t : threads) t.join();

```

A ce stade le solde du compte est un int.

jackpot.cpp

```

1 int main () {
2     vector<thread> threads;
3     threads.reserve(NB_THREAD);
4     Compte c;
5     for (int i=0; i < NB_THREAD ; i++) {
6         // std::ref pour garantir que "c" existera plus longtemps que les threads.
7         threads.emplace_back(jackpot, std::ref(c));
8     }
9     for (auto & t : threads) {
10         t.join();
11     }
12     if (c.getSolde() == N*JP) {
13         cout << "Dragon dort sur trésor " << c.getSolde() << endl;
14         return 0;
15     } else {
16         cout << "Ma cassette, mon sang !" << c.getSolde() << endl;
17         return 1;
18     }
19 }

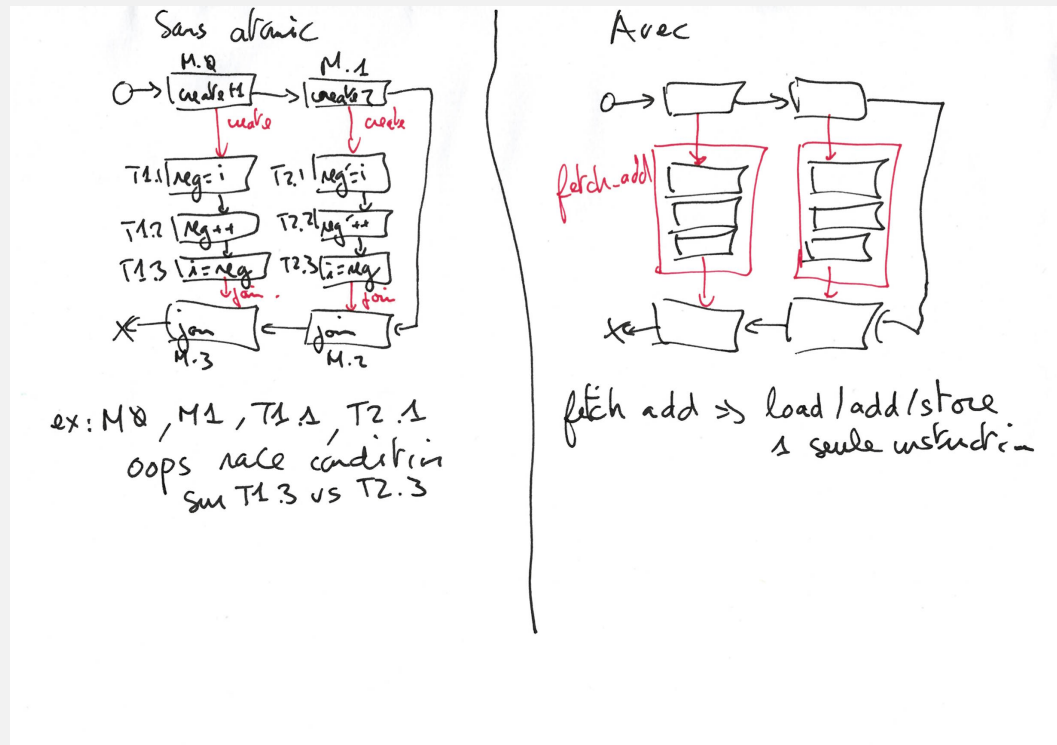
```

Question 5. Quel sera le solde du compte à la fin du programme pour $N=10$?

Ben ça marche pas bien. On manque d'ordre/contraintes sur les actions des threads. Du coup le résultat est mal déterminé.

Ce dessin explique le problème. On doit décomposer l'incrément du solde en 3 instructions : load, add, store en pratique. On peut donc exhiber des traces problématiques, dès que les deux thread commencent à travailler en même temps (i.e. se passent la main alors qu'ils sont dans le traitement).

On a donc assez probablement un solde inférieur à ce qu'il devrait être.



Question 6. Si le nombre de pièces d'or du jackpot est faible (disons 100), on n'observera pas de problème en général sur cet exemple, expliquez pourquoi.

C'est lié aux causes possibles de commutation : E/S, sleep ou yield explicite, synchro, épuisement de quantum.

Ici c'est l'épuisement de quantum la source de commutation intempestive principale, donc si compter 100 pièces prend moins d'un quantum, vu qu'on ne fait que ça, peu de chance de commuter.

Le programme reste faux (!)

On refuse dans l'UE e.g. les synchro avec des sleep pour "attendre" les autres. Ou de compter sur des effets dus au quantum/hypothèses en général sur le scheduler.

Ici en multi core, on peut encore observer des effets mauvais, malgré le temps court d'exécution.

Question 7. Comment utiliser un `atomic` pour corriger ces problèmes ? Expliquez l'effet au niveau des entrelacements possibles.

Donc on utilise `atomic<int>` pour déclarer le solde du compte

jackpot.cpp

```

1 #include <thread>
2 #include <iostream>
3 #include <atomic>
4
5 using namespace std;
6
7 class Compte {
8     atomic<int> solde;
9 public :
10     Compte(int solde=0):solde(solde) {}
11     void crediter (size_t val) { solde+=val ;}
12     int getSolde() const {return solde;}
13 };

```

Si créditer utilise bien `+=` pour ajouter au solde, le reste du code n'est pas modifié. Utiliser `solde = solde + val;` est incorrect par contre.

Le dessin du corrigé de la question précédente (à droite) explique le résultat. Les trois actions load/add/store sont groupées en une seule atomiquement (au niveau hardware).

Sans avoir ajouté d'ordre (synchro) entre les instructions des deux thread, on parvient quand même à écarter les exécutions problématiques du cas sans protection.

1.3 Section Critique.

On ajoute au Compte de la question précédente une opération : `bool debiter (int val)` qui doit contrôler que le solde du compte est suffisant (la banque ne prête pas), et si c'est le cas réduire le solde du montant indiqué. La fonction rend vrai si le débit a eu lieu.

```

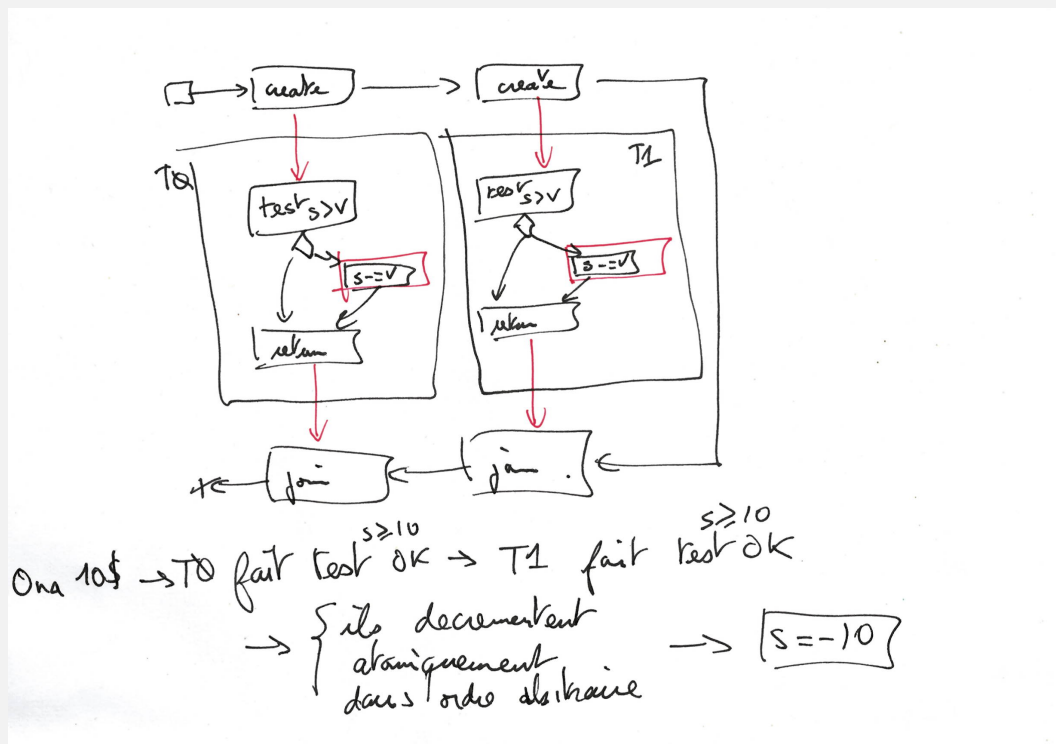
1 class Compte {
2     ...
3     bool debiter (unsigned int val) {
4         bool doit = (solde >= val);
5         if (doit) {
6             solde-=val ;
7         }
8         return doit;
9     }
10    ...
11 };
12 void losepot(Compte & c) {
13     for (int i=0; i < JP / 10; i++)
14         c.debiter(10);
15 }

```

Question 8. On reprend l'exemple du Jackpot, mais cette fois-ci en débit, le LosePot retire 10000 par paquets de 10. On lance N thread qui exécutent cette fonction ; le compte peut-il tomber en négatif ? Quelle garantie fournit `atomic` ici ?

Attention à la version de debit fournie, elle a une forme un peu bizarre (un seul return...), pour faciliter les dessins et l'introduction du mutex ensuite.

Oui, on tombe joyeusement en négatif. On peut atteindre au plus $-10 * NB_THREAD$.



Le test et sa mise à jour ne sont pas atomiques.

atomic fournit toujours une garantie sur les exécutions de `solde- = val`, donc on aura une valeur cohérente dans `solde` à la fin avec le nombre de fois où débiter à rendu `true`.

Question 9. Introduire un mutex dans la classe `Compte` pour sécuriser son utilisation dans un contexte Multi-Thread.

losepotmutex.cpp

```

1 class Compte {
2     mutable mutex m;
3     int solde;
4 public :
5     Compte(unsigned int solde=0):solde(solde) {}
6     void crediter (unsigned int val) {
7         m.lock();
8         solde+=val ;
9         m.unlock();
10    }
11    bool debiter (unsigned int val) {
12        m.lock();
13        bool doit = solde >= val;
14        if (doit) {
15            solde-=val ;
16        }
17        m.unlock();

```

```

18         return doit;
19     }
20     int getSolde() const {
21         m.lock();
22         auto ret = solde;
23         m.unlock();
24         return ret;
25     }
26 };

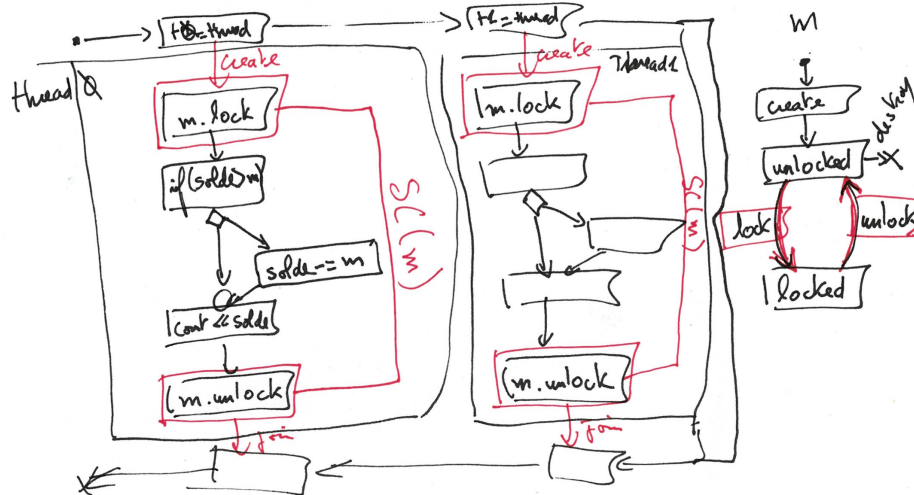
```

Sur le dessin, on a les deux thread de d'habitude, mais aussi le lock, qui a son propre état interne.

Quand on franchit l'action lock/unlock dans le thread, simultanément il faut franchir l'arc rouge du mutex.

NB: c'est le même thread qui doit faire lock et unlock avec des mutex. cf. Sémaphores pour des synchros où l'on débloquent l'autre thread.

On voit aussi ici qu'on doit déclarer le mutex comme étant "mutable", un nouveau mot clé pour contourner la nature "const" des accesseurs en lecture.



→ état du mutex séparé ⇒ partagé -

Question 10. Utiliser un `unique_lock` plutôt que l'API `lock/unlock`. Comparer les syntaxes.

Donc la création du `unique_lock` va faire directement un lock sur le mutex (sauf si on demande `std::defer`). Sa destruction (i.e. son destructeur) fait le unlock automatiquement.

Il protège donc le bloc dans lequel il est déclaré : comme c'est une variable ordinaire, il est stack alloc, et sera détruit en sortant du bloc.

Il évite donc d'oublier les unlock, particulièrement quand la structure de contrôle est complexe (e.g.

plusieurs “return”, exceptions, ...).

Sur la version de travail de debiter, ça ne change pas grand chose, mais si on compare debiter2 et debiter3 du corrigé, le mécanisme prend tout son intérêt.

On peut faire le rapprochement avec le bloc “synchronized” de Java.

losepotguard.cpp

```

1  class Compte {
2      // mutable contre le getSolde() qui est const
3      mutable mutex m;
4      int solde;
5  public :
6      Compte(int solde=0):solde(solde) {}
7      void crediter (unsigned int val) {
8          unique_lock<mutex> g(m);
9          solde+=val ;
10     }
11     bool debiter (unsigned int val) {
12         unique_lock<mutex> g(m);
13         bool doit = solde >= val;
14         if (doit) {
15             solde-=val ;
16         }
17         return doit;
18     }
19     bool debiter2 (unsigned int val) {
20         unique_lock<mutex> g(m);
21         if (solde >= val) {
22             solde-=val ;
23             return true;
24         } else {
25             return false;
26         }
27     }
28     bool debiter3 (unsigned int val) {
29         m.lock();
30         if (solde >= val) {
31             solde-=val ;
32             m.unlock();
33             return true;
34         } else {
35             m.unlock(); // ne pas oublier !!
36             return false;
37         }
38     }
39     int getSolde() const {
40         unique_lock<mutex> g(m);
41         return solde;
42     }
43 };

```

Question 11. Dans les versions avec un mutex, qu’apporte l’utilisation d’un atomic pour l’attribut solde ? Si l’on n’utilise pas `atomic` est-ce nécessaire de protéger la méthode `getSolde` avec le mutex ?

Si le mutex protège déjà les accès au solde, atomic semble ne pas apporter grand chose. La seule chose qu'il garantit c'est les load/store atomic à ce stade : le mutex protège déjà.

Si getSolde n'est pas protégé, a priori, ça ne devrait rien changer : On fait return solde, on lira la valeur avant ou après une écriture (dans tous les cas ce sera OK a priori).

En réalité on est déjà dans du Undefined Behavior ! Il est interdit de lire et écrire à la même adresse simultanément !

C'est ok si : "IF you're reading/writing 4-byte value AND it is DWORD-aligned in memory AND you're running on the I32 architecture, THEN reads and writes are atomic."

En général c'est du UB en C++, atomic fera ce qu'il faut faire sur votre archi matérielle.

Si le solde était un "long double" e.g. qui fasse plus qu'un mot mémoire, on voit mieux les problèmes ! Donc c'est une faute de lire pendant que quelqu'un y écrit (on peut lire la moitié de l'écriture de l'autre).

Il faudrait donc remettre de l'atomic ! Mais vu qu'on a construit un mutex par Compte, autant aller au bout de l'approche, la version mutex n'a pas besoin d'utiliser atomic, mais doit bien protéger tous ses accès avec le mutex, lecture et écriture.

Cela nous donne une forme générale pour une classe MT-safe sans trop réfléchir, similaire à ce qu'on a en annotant une classe Java par "synchronized" sur chaque méthode.

Question 12. La classe `Compte` en l'état ne dispose pas de constructeur par copie : mutex n'est pas copiable, ce qui rend la version générée par le compilateur de ce constructeur par copie non disponible. Comment écrire ce constructeur ?

NB: Sans constructeur par copie, on ne peut par exemple pas insérer un `Compte` dans un `std::vector`.

Donc mutex n'est pas copiable, ce qui rend la version générée par le compilateur du ctor par copie non disponible. Il faut locker l'objet source pour faire la copie.

transferts.cpp

```
1 // NB : vector exige Copyable, mais mutex ne l'est pas...
2 Compte(const Compte & other) {
3     other.lock();
4     solde = other.solde;
5     other.unlock();
6 }
```

TME 4 : Mutex et section critique

Objectifs pédagogiques :

- atomic, mutex
- sections critique, interblocage

1.1 Mise en place

On rappelle la classe `Compte` obtenue à la fin du TD 3.

```
1  class Compte {
2      mutable mutex m;
3      int solde;
4  public :
5      Compte(int solde=0):solde(solde) {}
6      void crediter (unsigned int val) {
7          unique_lock<mutex> g(m);
8          solde+=val ;
9      }
10     bool debiter (unsigned int val) {
11         unique_lock<mutex> g(m);
12         bool doit = solde >= val;
13         if (doit) {
14             solde-=val ;
15         }
16         return doit;
17     }
18     int getSolde() const {
19         unique_lock<mutex> g(m);
20         return solde;
21     }
22     // NB : vector exige Copyable, mais mutex ne l'est pas...
23     Compte(const Compte & other) {
24         other.m.lock();
25         solde = other.solde;
26         other.m.unlock();
27     }
28 };
```

1.2 Transaction.

On considère à présent une Banque, possédant en attribut un ensemble de K comptes (un `vector<Compte>`) initialement avec un solde de *SOLDEINITIAL* chacun. Elle est munie d'une opération `bool transfer(int idDebit, int idCredit, size_t val)` qui essaie de débiter le compte d'indice *idDebit* de *val*, et si c'est un succès, crédite le compte *idCredit* du même montant *val*. La fonction rend vrai si le transfert est un succès.

transferts.cpp

```
1  class Banque {
2      typedef vector<Compte> comptes_t;
3      comptes_t comptes;
4  public :
5      Banque (size_t ncomptes, size_t solde) : comptes (ncomptes, Compte(solde)){
6      }
7      void transfert(size_t deb, size_t cred, unsigned int val) {
8          Compte & debiteur = comptes[deb];
```

```

9      Compte & crediteur = comptes[cred];
10      if (debiteur.debiter(val)) {
11          crediteur.crediter(val);
12      }
13  }
14  size_t size() const {
15      return comptes.size();
16  }
17 };

```

Question 1. Ecrivez un programme, qui crée N threads de transaction, qui bouclent 1000 fois sur le comportement est suivant :

- Choisir i et j deux indices de comptes aléatoires, et un montant aléatoire m compris entre 1 et 100.
- Essayer de transférer le montant m de i à j .
- Dormir une durée aléatoire de 0 à 20 ms.

Fonction exécutée par les threads :

transferts.cpp

```

1 void transfertJob (int index, Banque & banque) {
2     std::cout << "started "<< index << endl;
3     for (int i= 0 ; i < 100000; i++) {
4         int debite = rand() % banque.size();
5         int credite = rand() % banque.size();
6         int val = rand() % 70 + 30;
7         banque.transfert(debite,credite,val);
8         //std::this_thread::sleep_for (std::chrono::milliseconds(rand()%10));
9     }
10    std::cout << "finished "<< index << endl;
11 }

```

Et le main :

transferts.cpp

```

1 int main () {
2     using namespace pr;
3     const int N = 10;
4     ::srand(::time(nullptr));
5     vector<thread> threads;
6     threads.reserve(N);
7
8     Banque b(200,100);
9
10    for (int i=0; i < N ; i++) {
11        // std::ref pour passer au thread par référence
12        threads.emplace_back(transfertJob,i,std::ref(b));
13        std::cout << "created "<< i << endl;
14    }
15    for (int i=0; i < N ; i++) {
16        threads[i].join();
17        std::cout << "joined "<< i << endl;
18    }
19
20    return 0;
21 }

```

Question 2. Le comportement est-il correct (pas de *datarace*) avec les protections actuelles sur le Compte ?

A priori oui, notre classe Compte est MT-safe, on peut s'en servir dans n'importe quel contexte MT sans risquer de fautes. D'où l'intérêt des classes de librairie MT safe.

On estime que découpler les débits des crédits est une faute, on souhaite au contraire que la mise à jour des deux comptes concernés soit atomique : soit le transfert a lieu et les deux comptes sont mis à jour (simultanément du point de vue d'un observateur), soit il n'a pas lieu. A aucun moment un observateur ne doit pouvoir voir un des comptes débités et l'autre pas encore crédité.

Question 3. Pour permettre de manipuler le mutex des comptes dans ce scenario, on propose d'ajouter un accesseur `mutex & getMutex()` au compte pour permettre de l'écrire, ou de définir les trois méthode `void lock () const`, `void unlock() const`, `bool try_lock () const` par délégation sur le mutex stocké pour implanter le contrat d'un *Lockable* C++¹.

La solution avec les trois méthodes permet de respecter le contrat d'un Lockable (mutex), i.e. on peut passer des comptes à `std::lock` la version multiple.

Le corrigé adopte cette approche, donc on fera `compte.lock()` plutôt que `compte.getLock().lock()`.

Ajouts dans la classe Compte :

```

                                transferts.cpp
1      // accès au lock pour la banque
2      void lock () const {
3          m.lock();
4      }
5      void unlock() const {
6          m.unlock();
7      }
8      // pour satisfaire le contrat Lockable exigé par la fonction multi lock
9      bool try_lock () const {
10         return m.try_lock();
11     }

```

Question 4. Proposez une stratégie de synchronisation pour permettre ce comportement transactionnel au niveau de `transfert`.

Si l'on veut une transaction, il faut empêcher l'accès au compte débité jusqu'à ce que le compte crédité soit mis à jour.

Une solution possible est de prendre les deux locks sur les deux comptes, avant de commencer à faire des opérations, et les relacher à la fin de la transaction.

Donc conceptuellement :

- `debiteur.lock()`
- `crediteur.lock()`
- `if (debiteur.debiter(m)) crediteur.crediter(m);`
- `debiteur.unlock() ; crediteur.unlock();`

```

                                transferts.cpp
1      void transfert_deadlock(size_t deb, size_t cred, unsigned int val) {

```

¹cf. https://en.cppreference.com/w/cpp/named_req

```

2      Compte & debiteur = comptes[deb];
3      Compte & crediteur = comptes[cred];
4      debiteur.lock();
5      crediteur.lock();
6      if (debiteur.debiter(val)) {
7          crediteur.crediter(val);
8      }
9      debiteur.unlock();
10     crediteur.unlock();
11 }

```

Question 5. Le programme se bloque immédiatement, même avec un seul thread qui fait des transferts. Pourquoi ?

Par défaut on a utilisé un `mutex` qui ne permet pas de double lock, même si le même thread détient déjà le lock. Ce comportement est différent de celui eg. des barrières `synchronized` de Java.

Ici, on lock le compte, puis on essaie de faire des crédits/débites dessus, donc on reprend le même lock. C'est un deadlock assez trivial.

Pour permettre une réacquisition il nous faut un `recursive_mutex`. En dehors de la déclaration, l'utilisation est la même que le `mutex` "normal".

Un même thread peut le lock plusieurs fois, mais le nombre d'`unlock` et de `lock` doit rester cohérent. Souvent permet une programmation plus simple au niveau client de la classe.

C'est aussi la sémantique proposée en Java pour "`synchronized`".

Attention à mettre à jour le paramètre générique du unique lock

transferts.cpp

```

1  class Compte {
2      mutable recursive_mutex m;
3      int solde;
4  public :
5      Compte(int solde=0):solde(solde) {}
6      void crediter (unsigned int val) {
7          unique_lock<recursive_mutex> g(m);
8          solde+=val ;
9      }
10     bool debiter (unsigned int val) {
11         unique_lock<recursive_mutex> g(m);

```

Question 6. Après correction du problème précédent à l'aide de `recursive_mutex`, on introduit plusieurs thread faisant des transferts, mais de nouveau on observe parfois un interblocage, le programme entier se fige. Expliquez pourquoi et corriger le problème.

Donc deux threads T0 et T1 pour l'exemple.

T0 pioche la paire (i,j) et T1 pioche la paire (j,i).

T0 lock i

T1 lock j

T0 et T1 vont maintenant s'interbloquer

Tout autre thread essayant de manipuler i ou j va aussi s'échouer.

Pour corriger; la bonne approche est d'ordonner les prises de locks. Par exemple modifier l'acquisition :

- if ($i > j$) debiteur.lock(); crediteur.lock();
- else crediteur.lock(); debiteur.lock();

Avec un ordre total imposé sur l'ensemble des locks, plus de cycles de deadlock possibles quand on fait des multi-acquisitions.

Le système dispose d'un ordre total sur les mutex/lock ; la fonction *lock()* prend (en varargs, séparés par des virgules) un ensemble de locks, en respectant cet ordre.

On peut donc se contenter de :

- lock (debiteur.getLock(), crediteur.getLock())

et faire confiance au système.

transferts.cpp

```

1  void transfert_multilock(size_t deb, size_t cred, unsigned int val) {
2      Compte & debiteur = comptes[deb];
3      Compte & crediteur = comptes[cred];
4      lock(debiteur, crediteur);
5      if (debiteur.debiter(val)) {
6          crediteur.crediter(val);
7      }
8      debiteur.unlock();
9      crediteur.unlock();
10 }
11 void transfert_manuallock(size_t deb, size_t cred, unsigned int val) {
12     Compte & debiteur = comptes[deb];
13     Compte & crediteur = comptes[cred];
14     if (deb < cred) {
15         debiteur.lock();
16         crediteur.lock();
17     } else {
18         crediteur.lock();
19         debiteur.lock();
20     }
21     if (debiteur.debiter(val)) {
22         crediteur.crediter(val);
23     }
24     debiteur.unlock();
25     crediteur.unlock();
26 }
```

1.3 Comptabilité

On souhaite ajouter au programme des thread qui vérifient la comptabilité. On se donne une nouvelle opération *comptabiliser* dans la classe *Banque* :

BFL.cpp

```

1  bool comptabiliser (int attendu) const {
2      int bilan = 0;
3      int id = 0;
4      for (const auto & compte : comptes) {
5          if (compte.getSolde() < 0) {
6              cout << "Compte " << id << " en négatif : " << compte.getSolde() <<
7                  endl;
8          }
9          bilan += compte.getSolde();
10         id++;
11     }
12 }
```

```

11         if (bilan != attendu) {
12             cout << "Bilan comptable faux : attendu " << attendu << " obtenu : " <<
                bilan << endl;
13         }
14         return bilan == attendu;
15     }

```

Cette méthode itère sur les comptes en sommant leur solde, et vérifie que la somme vaut bien $SOLDEINITIAL * K$. Sinon elle lève une alerte sur la console et rend false.

Question 7. Un thread comptable qui tourne en concurrence avec les threads de transfert sera-t-il satisfait avec les synchronisations actuelles ? Expliquez pourquoi.

Question 8. Essayez d'observer une exécution où le comptable détecte une erreur (on pourra augmenter le nombre de tours de boucle, éliminer les sleep etc... pour favoriser l'apparition d'une faute).

Non, le thread comptable peut dépasser le compte i , puis un transfert se fait vers i depuis un compte que le comptable n'a pas encore atteint dans son itération. La compta sera déficitaire.

Le thread comptable a vraiment besoin de bloquer les modifications sur tous les comptes qu'il a déjà traités pendant qu'il travaille.

Question 9. Ajoutez un mutex dans la banque, et les synchronisations utiles pour que le thread comptable obtienne toujours les bons résultats.

Donc c'est l'approche Big Fat Lock.

comptabiliser est une méthode de la classe Banque, le mutex protège à la fois comptabiliser et l'opération de transfert.

comptableBFL.cpp

```

1
2  class Compte {
3      int solde;
4  public :
5      Compte(int solde=0):solde(solde) {}
6      void crediter (unsigned int val) {
7          solde+=val ;
8      }
9      bool debiter (unsigned int val) {
10         bool doit = solde >= val;
11         if (doit) {
12             solde-=val ;
13         }
14         return doit;
15     }
16     int getSolde() const {
17         return solde;
18     }
19 };
20
21 class BanqueBFL {
22     mutable mutex m;
23     typedef vector<Compte> comptes_t;
24     comptes_t comptes;
25 public :
26     BanqueBFL (size_t ncomptes, size_t solde) : comptes (ncomptes, Compte(solde)){
27     }

```



```

28 void transfert(size_t deb, size_t cred, unsigned int val) {
29     unique_lock<mutex> l(m);
30     Compte & debiteur = comptes[deb];
31     Compte & creditur = comptes[cred];
32     if (debiteur.debiter(val)) {
33         creditur.crediter(val);
34     }
35 }
36 size_t size() const {
37     unique_lock<mutex> l(m);
38     return comptes.size();
39 }
40 bool comptabiliser (int attendu) const {
41     unique_lock<mutex> l(m);
42     int bilan = 0;
43     int id = 0;
44     for (const auto & compte : comptes) {
45         if (compte.getSolde() < 0) {
46             cout << "Compte " << id << " en négatif : " << compte.getSolde()
47                 << endl;
48             bilan += compte.getSolde();
49             id++;
50         }
51         if (bilan != attendu) {
52             cout << "Bilan comptable faux : attendu " << attendu << " obtenu : "
53                 << bilan << endl;
54         }
55         return bilan == attendu;
56     }
57 };
58 void comptableJob (const BanqueBFL & banque, int iterations) {

```

Question 10. Avec un seul mutex dans la banque, la concurrence entre les thread de transfert n'est plus possible. De fait les mutex définis dans Compte ne servent plus à rien dans ce scenario. Proposez une autre approche qui réutilise les locks de Compte plutôt que de n'avoir qu'un seul lock. Indice : il faut juste empêcher les threads de transfert d'accéder aux comptes que le comptable a déjà vus, et crédités dans son bilan. Attention cependant à ne pas réintroduire de deadlocks.

Ben comptabiliser doit soit commencer par lock tous les comptes (violent), soit verrouiller tous les comptes qu'il a déjà comptabilisé.

Du coup, vu qu'on le fait à la main, il faut que l'ordre utilisé dans transfert soit congruent (i.e. l'ordre à la main, basé sur les indices de comptes est bien).

Surtout ne pas mélanger les deux, les ordres ne seraient pas cohérents menant à un deadlock.

comptableOpt.cpp

```

1 #include <thread>
2 #include <mutex>
3 #include <atomic>
4 #include <iostream>
5 #include <random>
6
7 using namespace std;
8

```

```

9
10 // Comptabilité et transferts
11 namespace exBanque {
12
13 class Compte {
14     // mutable contre le getSolde() qui est const
15     // recursive_mutex à la question 15
16     mutable recursive_mutex m;
17     int solde;
18 public :
19     Compte(int solde=0):solde(solde) {}
20     void crediter (unsigned int val) {
21         lock_guard<recursive_mutex> g(m);
22         solde+=val ;
23     }
24     bool debiter (unsigned int val) {
25         lock_guard<recursive_mutex> g(m);
26         bool doit = solde >= val;
27         if (doit) {
28             solde-=val ;
29         }
30         return doit;
31     }
32     int getSolde() const {
33         lock_guard<recursive_mutex> g(m);
34         return solde;
35     }
36     // accès au lock pour la banque
37     void lock () const {
38         m.lock();
39     }
40     void unlock() const {
41         m.unlock();
42     }
43     // NB : vector exige Copyable, mais mutex ne l'est pas...
44     Compte(const Compte & other) {
45         other.lock();
46         solde = other.solde;
47         other.unlock();
48     }
49 };
50
51 class Banque {
52     typedef vector<Compte> Comptes;
53     Comptes comptes;
54     const int initial;
55 public :
56     Banque (size_t ncomptes, size_t solde) : comptes (ncomptes, Compte(solde)),initial
57         (solde){
58     }
59     // version à la main de l'ordre
60     void transfert(size_t deb, size_t cred, unsigned int val) {
61         Compte & debiteur = comptes[deb];
62         Compte & crediteur = comptes[cred];
63         if (deb < cred) {
64             debiteur.lock();
65             crediteur.lock();
66         } else {
67             crediteur.lock();

```

```

67         debiteur.lock();
68     }
69     if (debiteur.debiter(val)) {
70         crediteur.crediter(val);
71     }
72     debiteur.unlock();
73     crediteur.unlock();
74 }
75 size_t size() const {
76     return comptes.size();
77 }
78 bool comptabiliser () const {
79     int attendu = initial * comptes.size();
80     int bilan = 0;
81     int id = 0;
82     for (const auto & compte : comptes) {
83         // NB ordre des locks congruent avec celui utilisé dans transfert
84         compte.lock();
85         if (compte.getSolde() < 0) {
86             cout << "Compte " << id << " en négatif : " << compte.getSolde()
87                 << endl;
88             bilan += compte.getSolde();
89             id++;
90         }
91         for (const auto & compte : comptes) {
92             compte.unlock();
93         }
94         if (bilan != attendu) {
95             cout << "Bilan comptable faux : attendu " << attendu << " obtenu : "
96                 << bilan << endl;
97         }
98         return bilan == attendu;
99     }
100 };
101 void comptableJob (const Banque & banque, int iterations) {
102     for (int iter = 0 ; iter < iterations ; iter++) {
103
104         if (! banque.comptabiliser()) {
105             cout << "Comptable fâché !!" << endl;
106         }
107         cout << "Bilan " << iter << " fini."<<endl;
108         std::this_thread::sleep_for (std::chrono::milliseconds(20));
109     }
110 }
111
112 void transfertJob (int index, Banque & banque) {
113     std::cout << "started "<< index << endl;
114     for (int i= 0 ; i < 100000; i++) {
115         int debite = rand() % banque.size();
116         int credite = rand() % banque.size();
117         int val = rand() % 70 + 30;
118         banque.transfert(debite,credite,val);
119         //std::this_thread::sleep_for (std::chrono::milliseconds(rand()%10));
120     }
121     std::cout << "finished "<< index << endl;
122 }
123 }

```

```
124 int createAndWait (int N) {
125     vector<thread> threads;
126     threads.reserve(N);
127
128     Banque b(200,100);
129
130     for (int i=0; i < N -2 ; i++) {
131         threads.emplace_back(thread(transfertJob,i,std::ref(b)));
132         std::cout << "created "<< i << endl;
133     }
134     for (int i= N -2 ; i < N ; i++) {
135         threads.emplace_back(thread(comptableJob,std::cref(b),10));
136         std::cout << "created "<< i << endl;
137     }
138     for (int i=0; i < N ; i++) {
139         threads[i].join();
140         std::cout << "joined "<< i << endl;
141     }
142
143     return 0;
144 }
145
146 }
147
148 int main16 () {
149     ::srand(::time(nullptr));
150     return exBanque::createAndWait(10);
151 }
```